

```
#!/usr/bin/env python3
```

```
"""
```

Detection Blueprint Compiler

Translates Detection-as-Code YAML/JSON templates into:

1. Sigma Rules (Standardized YAML)
2. Splunk Search Processing Language (SPL)
3. Microsoft Sentinel Kusto Query Language (KQL)

Designed with zero third-party dependencies (Pure Python Standard Library).

Supports native JSON input as well as a built-in clean YAML parser/emitter.

```
"""
```

```
import argparse
```

```
import json
```

```
import re
```

```
import sys
```

```
import uuid
```

```
from typing import Any, Dict, List, Tuple, Union
```

```
def parse_simple_yaml(text: str) -> Dict[str, Any]:
```

```
    """
```

Lightweight, deterministic parser for hierarchical key-value YAML detection templates.

Handles nested mappings, string scalar lists, and inline comments without external PyYAML.

```
    """
```

```
    lines = text.splitlines()
```

```
    root: Dict[str, Any] = {}
```

```
    stack: List[Tuple[int, Union[Dict[str, Any], List[Any]]]] = [(-1, root)]
```

```
    last_key: Dict[int, str] = {}
```

```
    for raw_line in lines:
```

```
        stripped = raw_line.strip()
```

```
        if not stripped or stripped.startswith("#"):
```

```
            continue
```

```
        # Count leading whitespace indentation
```

```
        indent = len(raw_line) - len(raw_line.lstrip(" "))
```

```
        content = stripped.split("#")[0].strip()
```

```
        # Unwind stack to find current parent scope
```

```
        while len(stack) > 1 and stack[-1][0] >= indent:
```

```
            stack.pop()
```

```

current_parent = stack[-1][1]

if content.startswith("- "):
    # List item
    val = content[2:].strip().strip("\'")
    if isinstance(current_parent, list):
        # Try parsing integers
        if val.isdigit():
            current_parent.append(int(val))
        else:
            current_parent.append(val)
elif ":" in content:
    key, val = content.split(":", 1)
    key = key.strip()
    val = val.strip().strip("\'")

    if val == "":
        # Could be parent of dict or list
        new_container: Dict[str, Any] = {}
        if isinstance(current_parent, dict):
            current_parent[key] = new_container
        elif isinstance(current_parent, list):
            current_parent.append({key: new_container})
        stack.append((indent, new_container))
        last_key[indent] = key
    else:
        # Leaf value
        if val.isdigit():
            final_val: Any = int(val)
        else:
            final_val = val
        if isinstance(current_parent, dict):
            current_parent[key] = final_val

```

# Post-process: convert empty child dicts whose children are lists into Python lists

```
def fix_empty_dict_to_list(d: Any) -> Any:
```

```

    if isinstance(d, dict):
        # check if items were actually parsed as sequential keys or lists
        cleaned = {}
        for k, v in d.items():
            cleaned[k] = fix_empty_dict_to_list(v)
        return cleaned

```

```
return d
```

```
# Re-parse robustly via line walker for detection structure  
return parse_detection_yaml_robust(text)
```

```
def parse_detection_yaml_robust(text: str) -> Dict[str, Any]:  
    """Robust recursive descent line parser tailored for detection engineering templates."""  
    lines = text.splitlines()  
  
    def parse_block(idx: int, current_indent: int) -> Tuple[Union[Dict[str, Any], List[Any]], int]:  
        obj: Dict[str, Any] = {}  
        arr: List[Any] = []  
        is_list = None  
  
        while idx < len(lines):  
            line = lines[idx]  
            stripped = line.strip()  
            if not stripped or stripped.startswith("#"):  
                idx += 1  
                continue  
  
            indent = len(line) - len(line.lstrip(" "))  
            if indent < current_indent:  
                break  
  
            clean_line = stripped.split("#")[0].rstrip()  
  
            if clean_line.startswith("- "):  
                if is_list is False:  
                    break  
                is_list = True  
                val = clean_line[2:].strip().strip("\"'")  
                if val.isdigit():  
                    arr.append(int(val))  
                elif val != "":  
                    arr.append(val)  
                else:  
                    # Nested block under list  
                    sub_obj, idx = parse_block(idx + 1, indent + 2)  
                    arr.append(sub_obj)  
                    continue  
            idx += 1
```

```

elif ":" in clean_line:
    if is_list is True:
        break
    is_list = False
    key, val = clean_line.split(":", 1)
    key = key.strip()
    val = val.strip().strip("\'")

    if val == "":
        # Peek next line indent
        sub_obj, idx = parse_block(idx + 1, indent + 1)
        obj[key] = sub_obj
        continue
    else:
        if val.isdigit():
            obj[key] = int(val)
        else:
            obj[key] = val
        idx += 1
else:
    idx += 1

```

```

return (arr if is_list else obj), idx

```

```

parsed, _ = parse_block(0, 0)
return parsed if isinstance(parsed, dict) else {}

```

```

def dump_yaml_dict(data: Any, indent_level: int = 0) -> str:
    """Zero-dependency YAML serialization emitter."""
    spacing = " " * indent_level
    output = []
    if isinstance(data, dict):
        for k, v in data.items():
            if isinstance(v, dict):
                output.append(f"{spacing}{k}:")
                output.append(dump_yaml_dict(v, indent_level + 1))
            elif isinstance(v, list):
                output.append(f"{spacing}{k}:")
                for item in v:
                    if isinstance(item, dict):
                        output.append(f"{spacing} -")
                        output.append(dump_yaml_dict(item, indent_level + 2))

```

```

        else:
            val_str = f"{item}" if any(c in str(item) for c in [":", " ", "#"]) else str(item)
            output.append(f"{spacing} - {val_str}")
    else:
        val_str = f"{v}" if any(c in str(v) for c in [":", " ", "#", "*"]) else str(v)
        output.append(f"{spacing}{k}: {val_str}")
return "\n".join(output)

```

class DetectionCompiler:

"""Compiles generic detection maps into Sigma, Splunk SPL, and Sentinel KQL."""

def \_\_init\_\_(self, spec: Dict[str, Any]):

```

    self.spec = spec
    self.metadata = spec.get("metadata", {})
    self.taxonomy = spec.get("taxonomy", {})
    self.data_sources = spec.get("data_sources", {})
    self.logic = spec.get("detection_logic", {})
    self.response = spec.get("response", {})

```

def to\_sigma(self) -> str:

"""Converts detection blueprint into Sigma rule YAML."""

```

logsource = {
    "category": self.data_sources.get("log_source", "process_creation"),
    "product": self.data_sources.get("platform", "windows"),
}

```

```

detection_block: Dict[str, Any] = {}

```

```

if "selection_fields" in self.logic:
    detection_block["selection"] = self.logic["selection_fields"]

```

```

if "filter_exceptions" in self.logic:
    detection_block["filter"] = self.logic["filter_exceptions"]

```

```

raw_condition = str(self.logic.get("condition", "selection_fields"))

```

```

sigma_cond = (
    raw_condition.replace("selection_fields", "selection")
    .replace("filter_exceptions", "filter")
)

```

```

detection_block["condition"] = sigma_cond

```

```

tags: List[str] = []

```

```

for tactic in self.taxonomy.get("tactics", []):
    code = str(tactic).split("-")[0].strip().lower()

```

```

        tags.append(f"attack.{code}")
    for tech in self.taxonomy.get("techniques", []):
        code = str(tech).split("-")[0].strip().lower()
        tags.append(f"attack.{code}")

    sigma_dict = {
        "title": self.metadata.get("title", "Untitled Detection"),
        "id": self.metadata.get("id", str(uuid.uuid4())),
        "status": self.metadata.get("status", "experimental"),
        "description": self.metadata.get("description", ""),
        "author": self.metadata.get("author", "SecOps"),
        "date": str(self.metadata.get("date", "")),
        "modified": str(self.metadata.get("modified", "")),
        "tags": tags,
        "logsource": logsource,
        "detection": detection_block,
        "level": str(self.response.get("severity", "medium")).lower(),
    }

    return dump_yaml_dict(sigma_dict)

def _parse_field_criteria(self, raw_key: str, val: Any) -> Tuple[str, str, Any]:
    """Extracts field name, operator modifier, and target values."""
    if "|" in raw_key:
        field, modifier = raw_key.split("|", 1)
    else:
        field, modifier = raw_key, "exact"
    return field, modifier, val

def to_splunk_spl(self) -> str:
    """Translates blueprint into Splunk Search Processing Language (SPL)."""
    log_source = self.data_sources.get("log_source", "sysmon")
    event_ids = self.data_sources.get("event_ids", [])

    base_filters = [f'sourcetype="{log_source}']
    if event_ids:
        if isinstance(event_ids, list) and len(event_ids) > 1:
            base_filters.append(f'EventCode IN ({', '.join(map(str, event_ids))})')
        elif isinstance(event_ids, list) and len(event_ids) == 1:
            base_filters.append(f'EventCode={event_ids[0]}')
        else:
            base_filters.append(f'EventCode={event_ids}')

```

```

def build_spl_clauses(mapping: Dict[str, Any]) -> List[str]:
    clauses = []
    if not isinstance(mapping, dict):
        return clauses
    for raw_k, v in mapping.items():
        field, mod, val = self._parse_field_criteria(raw_k, v)
        val_list = val if isinstance(val, list) else [val]

        sub_clauses = []
        for item in val_list:
            escaped_item = str(item).replace("'", "\'")
            if mod == "endswith":
                sub_clauses.append(f'{field}="{escaped_item}"')
            elif mod == "startswith":
                sub_clauses.append(f'{field}="{escaped_item}"')
            elif mod == "contains":
                sub_clauses.append(f'{field}="{escaped_item}"')
            else:
                sub_clauses.append(f'{field}="{escaped_item}"')

        if len(sub_clauses) > 1:
            clauses.append(f'({ " OR ".join(sub_clauses) })')
        elif sub_clauses:
            clauses.append(sub_clauses[0])
    return clauses

sel_clauses = build_spl_clauses(self.logic.get("selection_fields", {}))
filter_clauses = build_spl_clauses(self.logic.get("filter_exceptions", {}))

spl_parts = base_filters + sel_clauses
if filter_clauses:
    for f_clause in filter_clauses:
        spl_parts.append(f"NOT {f_clause}")

title = str(self.metadata.get("title", "")).replace("'", "\'")
severity = str(self.response.get("severity", "medium")).replace("'", "\'")
search_query = "index=* " + " ".join(spl_parts)

pipeline = (
    f'{search_query}\n'
    f'| eval rule_title="{title}", severity="{severity}"\n'
    f'| table _time, host, User, Image, CommandLine, rule_title, severity"
)

```

```
return pipeline
```

```
def to_sentinel_kql(self) -> str:
```

```
    """Translates blueprint into Microsoft Sentinel Kusto Query Language (KQL)."""
```

```
    platform = str(self.data_sources.get("platform", "windows")).lower()
```

```
    log_source = str(self.data_sources.get("log_source", "process_creation")).lower()
```

```
    if "process" in log_source or platform == "windows":
```

```
        table_name = "DeviceProcessEvents"
```

```
    elif "auditd" in log_source or platform == "linux":
```

```
        table_name = "Syslog"
```

```
    else:
```

```
        table_name = "SecurityEvent"
```

```
    lines = [table_name]
```

```
    # Field mapping dictionary: maps generic names to Microsoft Sentinel schema
```

```
    kql_field_map = {
```

```
        "Image": "FolderPath",
```

```
        "CommandLine": "ProcessCommandLine",
```

```
        "ParentImage": "InitiatingProcessFolderPath",
```

```
        "ParentCommandLine": "InitiatingProcessCommandLine",
```

```
        "User": "AccountName",
```

```
    }
```

```
def build_kql_conditions(mapping: Dict[str, Any], negate: bool = False) -> List[str]:
```

```
    kql_conds = []
```

```
    if not isinstance(mapping, dict):
```

```
        return kql_conds
```

```
    for raw_k, v in mapping.items():
```

```
        field, mod, val = self._parse_field_criteria(raw_k, v)
```

```
        target_field = kql_field_map.get(field, field)
```

```
        val_list = val if isinstance(val, list) else [val]
```

```
    sub_predicates = []
```

```
    for item in val_list:
```

```
        escaped_item = str(item).replace("'", "\'")
```

```
        if mod == "endswith":
```

```
            sub_predicates.append(f'{target_field} endswith "{escaped_item}"')
```

```
        elif mod == "startswith":
```

```
            sub_predicates.append(f'{target_field} startswith "{escaped_item}"')
```

```
        elif mod == "contains":
```

```
            sub_predicates.append(f'{target_field} contains "{escaped_item}"')
```

```

        else:
            sub_predicates.append(f'{target_field} == "{escaped_item}"')

        predicate_str = " or ".join(sub_predicates)
        if len(sub_predicates) > 1:
            predicate_str = f'({predicate_str})'

        if negate:
            kql_conds.append(f'not({predicate_str})')
        else:
            kql_conds.append(predicate_str)
    return kql_conds

sel_kql = build_kql_conditions(self.logic.get("selection_fields", {}), negate=False)
for cond in sel_kql:
    lines.append(f'| where {cond}')

filter_kql = build_kql_conditions(self.logic.get("filter_exceptions", {}), negate=True)
for cond in filter_kql:
    lines.append(f'| where {cond}')

title = str(self.metadata.get("title", "")).replace("'", "\'")
severity = str(self.response.get("severity", "medium")).replace("'", "\'")
lines.append(f'| extend RuleTitle = "{title}", Severity = "{severity}"')
lines.append('| project TimeGenerated, DeviceName, AccountName, ActionType,
ProcessCommandLine, InitiatingProcessCommandLine, RuleTitle, Severity')

return "\n".join(lines)

def main():
    parser = argparse.ArgumentParser(description="Detection YAML Compiler to Sigma, SPL,
and KQL.")
    parser.add_argument("file", nargs="?", help="Path to input YAML or JSON file")
    parser.add_argument("--format", choices=["all", "sigma", "spl", "kql"], default="all",
help="Output format target")
    args = parser.parse_args()

    sample_template = ""
    metadata:
    id: "4f7a5b32-9c12-4f88-8123-bc7e39a01f99"
    title: "Suspicious PowerShell Execution via Encoded Bypass"
    status: "test"

```

description: "Detects PowerShell execution with bypass flags and base64 encoded strings."  
author: "Detection Engineering Team"  
date: "2026-10-06"  
modified: "2026-10-06"

taxonomy:

framework: "MITRE ATT&CK"

tactics:

- "TA0002-Execution"

techniques:

- "T1059.001-PowerShell"

data\_sources:

platform: "windows"

log\_source: "process\_creation"

event\_ids:

- 1
- 4688

detection\_logic:

selection\_fields:

Image|endswith: "powershell.exe"

CommandLine|contains:

- "-enc"
- "-ExecutionPolicy Bypass"

filter\_exceptions:

ParentImage|endswith: "sccm\_client.exe"

condition: "selection\_fields and not filter\_exceptions"

response:

severity: "high"

"""

if args.file:

with open(args.file, "r") as f:

content = f.read()

if args.file.endswith(".json"):

data = json.loads(content)

else:

data = parse\_detection\_yaml\_robust(content)

else:

data = parse\_detection\_yaml\_robust(sample\_template)

```
compiler = DetectionCompiler(data)
```

```
if args.format in ("all", "sigma"):  
    print("=" * 60)  
    print("SIGMA RULE (YAML)")  
    print("=" * 60)  
    print(compiler.to_sigma())  
    print()
```

```
if args.format in ("all", "spl"):  
    print("=" * 60)  
    print("SPLUNK SEARCH PROCESSING LANGUAGE (SPL)")  
    print("=" * 60)  
    print(compiler.to_splunk_spl())  
    print()
```

```
if args.format in ("all", "kql"):  
    print("=" * 60)  
    print("MICROSOFT SENTINEL (KQL)")  
    print("=" * 60)  
    print(compiler.to_sentinel_kql())  
    print()
```

```
if __name__ == "__main__":  
    main()
```